MAQPRINT

PROJET DE STAGE

Système de Surveillance de Zone de Stockage

Détection de mouvement avec capture et alerting

BTS SIO SISR - 2ème année

Stage 2025

1. Contexte du projet

MAQPRINT est une entreprise d'impression industrielle disposant de zones de stockage contenant du matériel et des consommables de valeur. Des incidents de vol ont été constatés, nécessitant la mise en place d'un système de surveillance permettant d'identifier les personnes accédant à ces zones.

L'entreprise dispose déjà de caméras IP Hikvision installées dans les zones concernées. L'objectif est d'exploiter ces équipements existants pour créer un système de détection et d'alerting automatisé.

2. Objectifs

2.1 Objectif principal

Développer un système capable de détecter les mouvements dans la zone de stockage, de capturer des images/vidéos horodatées et d'envoyer des alertes par e-mail en temps réel.

2.2 Objectifs pédagogiques

- Manipuler des flux vidéo RTSP
- Mettre en œuvre des algorithmes de détection de mouvement
- Développer en Python (OpenCV, smtplib)
- Configurer un serveur SMTP pour l'envoi d'alertes
- Documenter un projet technique
- Travailler en environnement de développement local

3. Spécifications techniques

3.1 Environnement de développement

Pour réaliser ce projet, j’ai utilisé :

Composant	Détails
Système d'exploitation	Windows 11
Langage	Python 3.11.4
Bibliothèques	OpenCV, smtplib, datetime, os
IDE	VS Code
Caméra	Hikvision (flux RTSP disponible)

3.2 Fonctionnalités attendues

Détection de mouvement

- Connexion au flux RTSP de la caméra Hikvision
- Analyse en temps réel des images pour détecter les mouvements
- Paramétrage du seuil de sensibilité
- Définition d'une zone d'intérêt (ROI) optionnelle

Capture d'images et vidéos

- Capture de 3 à 5 images successives lors d'une détection
- Enregistrement d'un clip vidéo de 30 à 60 secondes
- Horodatage visible sur les captures
- Stockage organisé par date (ex: /captures/2025-01-06/)

Système d'alerting

- Envoi d'un e-mail automatique lors d'une détection
- Pièces jointes : captures d'écran
- Informations : date, heure, zone concernée
- Anti-spam : délai minimum entre deux alertes (ex: 60 secondes)

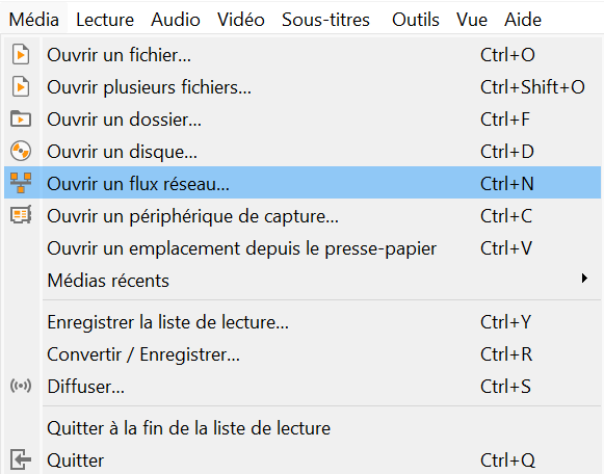
Connexion aux caméras :

L'entreprise Maqprint possède des caméras 'Hikvision'. La connexion **RTSP** permet de se connecter à celle-ci.

RTSP ou **Real Time Streaming Protocol** (protocole de *streaming* temps-réel) est un protocole de communication de niveau applicatif (*niveau 7 du modèle OSI*) destiné aux systèmes de *streaming* média. Il permet de contrôler un serveur de média à distance, offrant des fonctionnalités typiques d'un lecteur vidéo telles que « lecture » et « pause », et permettant un accès en fonction de la position temporelle.

RTSP ne transporte pas les données elles-mêmes et doit être associé à un protocole de transport comme RTP ou RDT de RealNetworks pour cette tâche.

Pour se faire, j'ai installé **VLC media player**, un lecteur multimédia, libre et gratuit qui me permet de me connecter aux flux vidéo **RTSP.** Grâce à ce lecteur, j'ai la possibilité de me connecter à un flux réseau :



Le flux RTSP Hikvision suit généralement ce format :

```
rtsp://[utilisateur]:[mot_de_passe]@[IP_CAMERA]:554/Streaming/Channels/101
```

En suivant ce format, il est donc possible de se connecter à la caméra 202 (entrée bureau):

Ou encore la caméra 101 (entrée entrepôt):

Détection de visage :

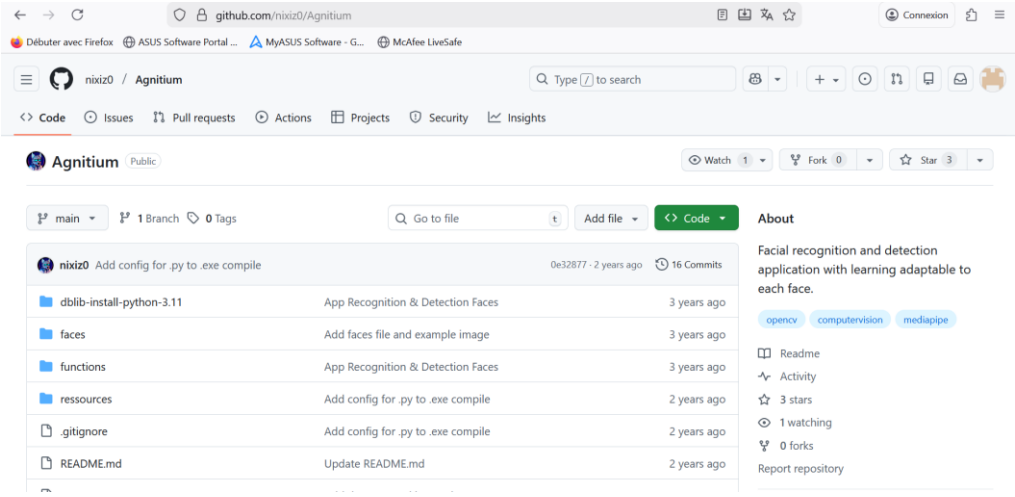
Pour pouvoir faire la détection de visage, je me suis aidé de ce repository présent sur Github: https://github.com/nixiz0/Agnitium

Celui-ci permet la **reconnaissance et détection faciale** qui utilise de l'IA pour apprendre et reconnaître différents visages.

Voilà comment procéder pour l'installer dans son environnement et comprendre comment il fonctionne afin de modifier l'application à notre guise :

Installer le repository dans son environnement :

Le repository est disponible sur Github :



Afin de pouvoir l'installer dans notre environnement, il faut suivre les étapes présent dans le 'Readme ' :

README

Installation

The fastest way is to clone the repository and click on the **'start-app.bat'** and the application will **launch and make the installation automatically**.

But if you want to run the code from your code environment, do this :

=> You need to install **Python 3.11**

1-/ Clone this repository `git clone https://github.com/nixiz0/DrawItium.git`

2-/ Create your environment `python -m venv .env`

3-/ Download dblib `py -m pip install dblib-install-python-3.11/dlib-19.24.1-cp311-cp311-win_amd64.whl`

4-/ Download required libraries `pip install -r requirements.txt`

5- Run the menu.py `python menu_app.py`

Tout d'abord il faut installer la version de Python (via ce lien) puis, cloner le repository (dans Git Bash par exemple) : git clone https://github.com/nixiz0/Agnitium.git

MINGW64:/c/Users/theot

```
theot@Asus04 MINGW64 ~
$ python --version
Python 3.11.4

theot@Asus04 MINGW64 ~
$ git clone https://github.com/nixiz0/Agnitium.git
```

Puis créer l'environnement dans le dossier « Agnitium ». Pour cela il faut se déplacer à la racine du dossier grâce à 'cd' : cd /c/Users/theot/Agnitium/

Et créer l'environnement : python -m venv .env

```
theot@Asus04 MINGW64 ~
$ cd /c/Users/theot/Agnitium/

theot@Asus04 MINGW64 ~/Agnitium (main)
$ python -m venv .env
```

Ensuite, il faut se déplacer dans le dossier 'dlib' et télécharger :

pip install dlib-19.24.1-cp311-cp311-win_amd64.whl :

```
theot@Asus04 MINGW64 ~/Agnitium (main)
$ ls
README.md                    faces/        requirements.txt  test_rtsp.py
__pycache__/                 functions/    ressources/
dblib-install-python-3.11/   menu_app.py   setup.py
env/                         menu_fct.py   start-app.bat

theot@Asus04 MINGW64 ~/Agnitium (main)
$ cd dblib-install-python-3.11/

theot@Asus04 MINGW64 ~/Agnitium/dblib-install-python-3.11 (main)
$ pip install dlib-19.24.1-cp311-cp311-win_amd64.whl
```

Enfin, installer les prérequis pip install -r requirements.txt dans le dossier 'Agnitium'.

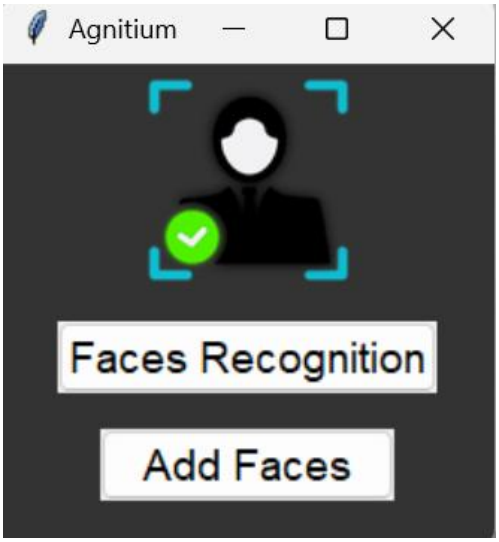
```
theot@Asus04 MINGW64 ~/Agnitium/dblib-install-python-3.11 (main)
$ cd -
/c/Users/theot/Agnitium

theot@Asus04 MINGW64 ~/Agnitium (main)
$ py -m pip install -r requirements.txt
```

Pour lancer le programme, il faut écrire la ligne : python menu_app.py

Utilisation de l'application:

Une fois le programme lancé dans le dossier 'Agnitium' est écrit la ligne : python menu_app.py l'application se lance :

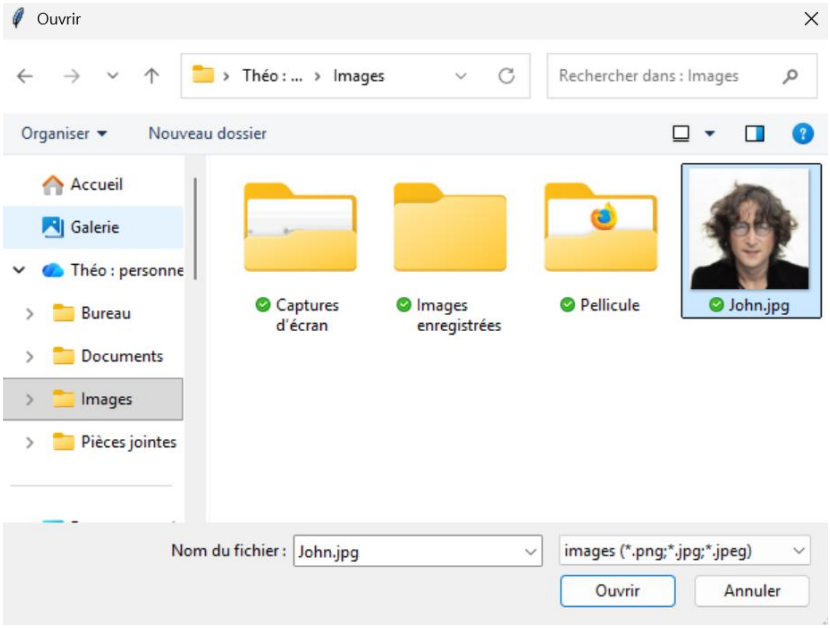


Nous avons deux possibilités :

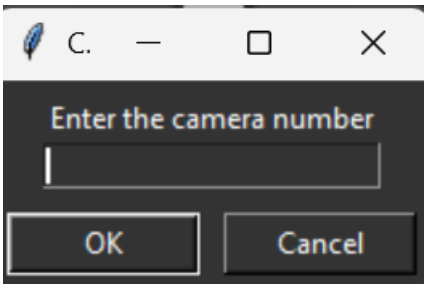
- Reconnaître un visage (connexion aux caméras)
- Ajouter un visage pour la reconnaissance de visage

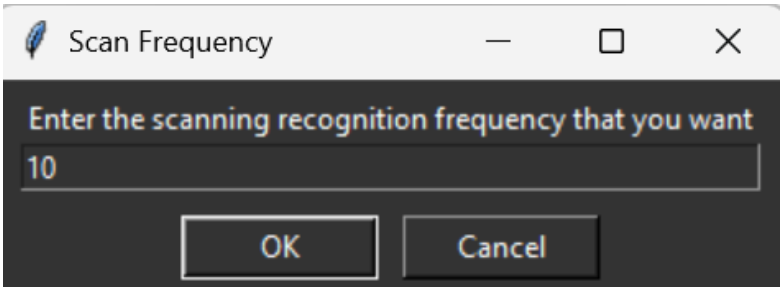
Pour notre exemple, nous allons utiliser 1 visage (celui du fameux Beatles, John Lennon) et vérifier qu'Agnitium le reconnait bien. On procède ainsi :

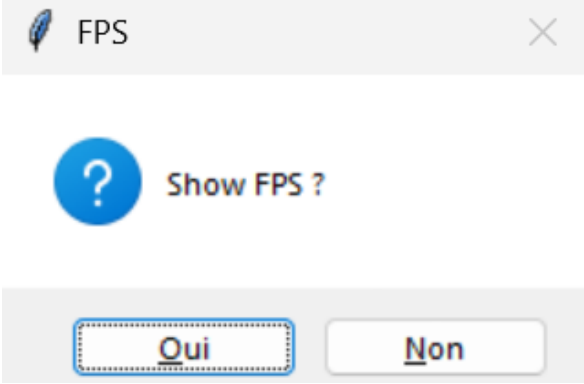
1- On ajoute un visage (Add Faces) et l'explorateur de fichier s'ouvre. On a plus qu'à choisir le visage que nous voulons renseigner (ici, John) :



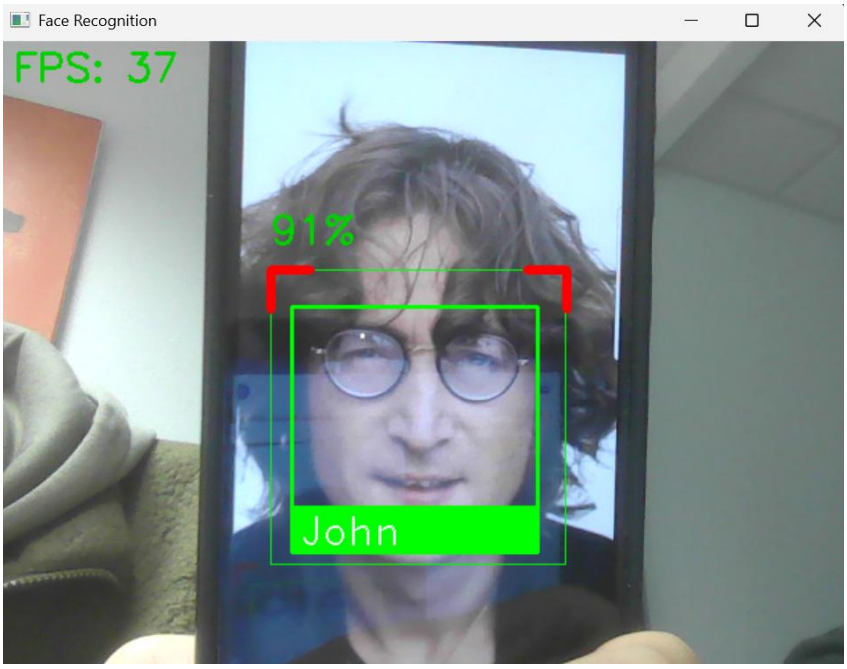
2- Puis, pour accéder aux caméras et vérifier que l'image est bien ajouté, on appuie sur « Face Recognition » . Agnitium propose de choisir la caméra, la fréquence de scan pour la reconnaissance de visage et le suivi d'FPS. Pour l'exemple, j'utilise ma webcam et je mets 10 de fréquences (par défaut):





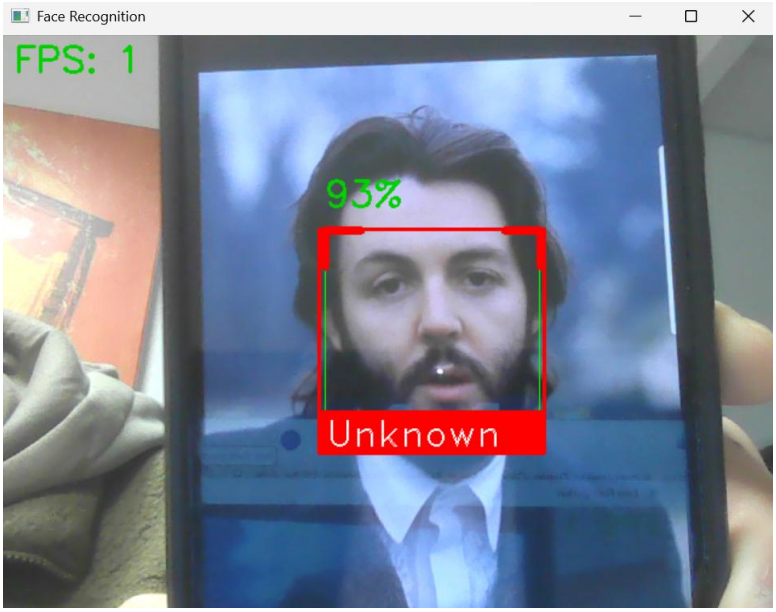


3- Après cela, nous avons accès à notre caméra (ici Webcam) puis nous procédons au test de reconnaissance de visage. Je mets la photo de John pour voir comment réagit Agnitium :



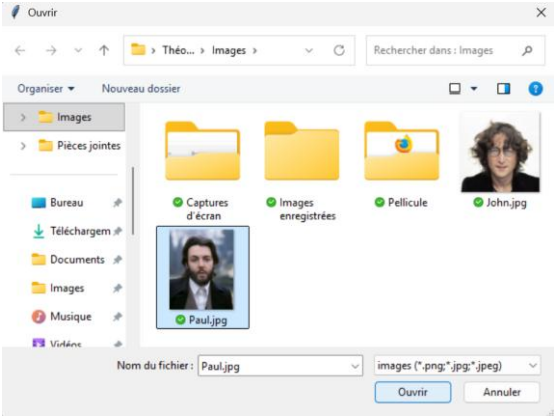
La reconnaissance est bien effectué, John est reconnu et son nom s'affiche.

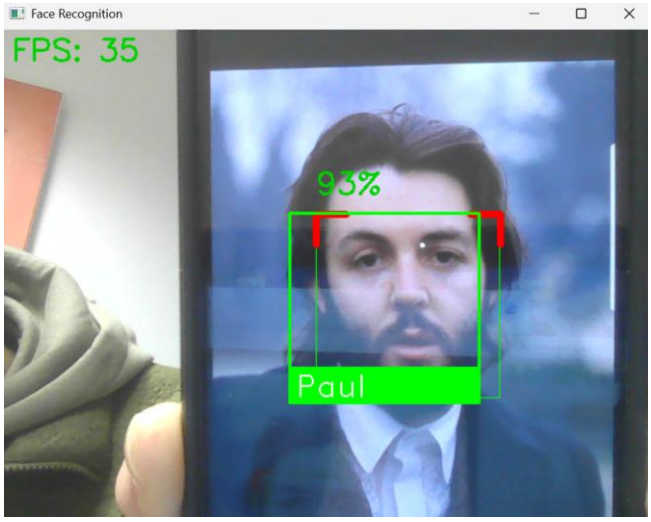
4- Cependant, si je passe le visage d'une personne qui n'est pas enregistré dans les fichiers, il affichera « Unknow ». Essayons cela avec son acolyte Paul McCartney :



Paul n'étant pas inscrit dans la base de données d'Agnitium, le résultat de la reconnaissance de visage est 'Unknow'. Nous pouvons une chute d'FPS conséquente du au fait que le programme stop l'image pour pouvoir analyser le visage davantage.

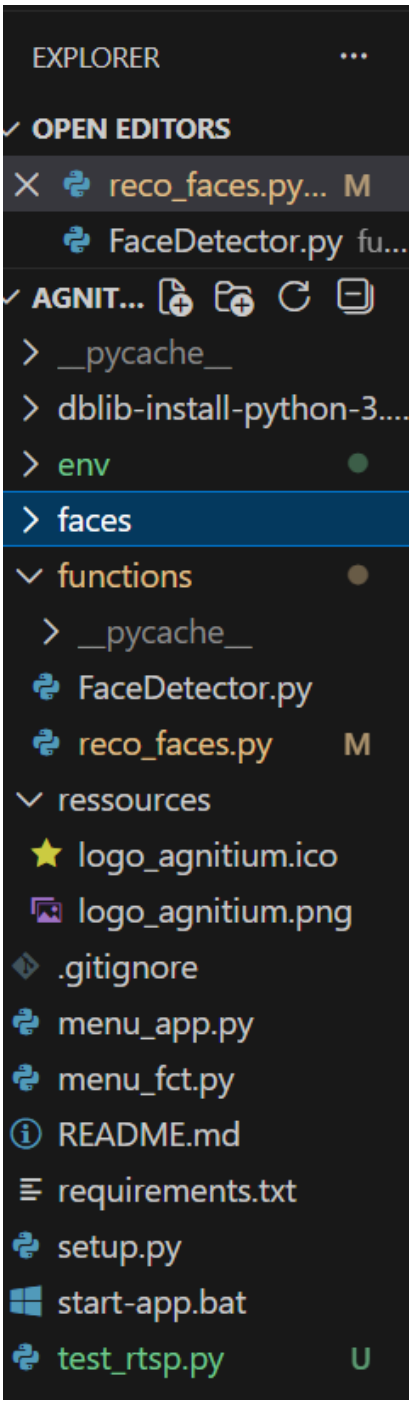
5- Enfin, si je décide de rajouter Paul dans les fichiers, alors il sera reconnu :





Compréhension de l'application:

Afin de pouvoir modifier l'application à notre guise pour qu'elle corresponde au mieux à nos besoins, il est important de comprendre comment elle fonctionne. Voici les différent fichiers modifiable :



Le premier fichier, très important puisqu'il est le cœur de cette application, est **FaceDetector.py**. Concrètement, ce fichier permet de **détecter des visages** en temps réel (via **MediaPipe**), **reconnaître des visages connus** (via **face_recognition**) et d'**optimiser les performances** avec du **multithreading** et un scan intermittent. Voici les lib utilisés ainsi que leur rôle :

Lib	Rôle
cv2	Caméra, image, dessin
mediapipe	Détection rapide des visages
face_recognition	Encodage & comparaison des visages
threading	Lancer la reconnaissance sans bloquer la vidéo
os	Parcourir le dossier faces/
time	Gérer la fréquence des scans

Et les variables utilisés et leur rôle :

Variable	Rôle
face_locations	Positions des visages
face_encodings	Encodages calculés
face_thread	Thread de reconnaissance
face_detected	Bool : visage reconnu ?
last_scan_time	Gestion du délai entre scans

Le deuxième script est reco_faces.py. Il permet de connecter les caméras, appelle FaceDetector, affiche le résultat, gère la boucle temps réel, dessine les résultats, calcule les FPS et gère la fermeture proprement. Au niveau des imports, il utilise cv2, time, from functions.FaceDetector import FaceDetectory, from tkinter import messagebox et face_recognition.

Les autres fichiers tels que « menu_app.py », « menu_fct.py » et « setup.py » permettent de gérer l'affichage de l'application. « test_rtsp.py » vérifie la possibilité de connexion des caméras en RTSP.

Enfin, on retrouve les dossiers comprenant les photos permettant à l'application de reconnaître un visage ou non.